

Unit 1 : Introduction

1. What is Python?

- Python is a **high-level, interpreted programming language**.
- Created by **Guido van Rossum** in **1991**.
- Designed to be **easy to read, write, and learn**.
- Famous for its **simple syntax**, like writing English.
- Used in **web development, data science, AI, machine learning, automation, software development, etc.**

2. History and Evolution of Python

- **1980s**: Guido van Rossum started working on Python at CWI (Netherlands).
- **1991**: First official release of Python (v0.9.0).
- **2000**: Python 2.0 released (introduced garbage collection, Unicode support).
- **2008**: Python 3.0 released (major changes, not backward-compatible).
- **Present**: Python 3 is the standard version, widely used in all industries.

The **latest** major stable release of Python is **Python 3.13.7**, released on August 14, 2025

3. Characteristics of Python

- **Simple and Easy** – Easy to learn, read, and write.
- **Interpreted** – Code runs line by line (no compilation needed).
- **Cross-Platform** – Works on Windows, Mac, Linux.
- **Object-Oriented** – Supports classes and objects.
- **Extensive Libraries** – Rich built-in modules (math, random, datetime, etc.).
- **Free and Open-Source** – Anyone can use and contribute.
- **High Productivity** – Less code, more output.

4. Advantages of Python

- Easy to learn and use.
- Fewer lines of code required.
- Huge community support.
- Rich libraries and frameworks (Django, Flask, NumPy, Pandas, TensorFlow).
- Portable and cross-platform.
- Great for beginners as well as professionals.

5. Disadvantages of Python

- Slower than C/C++ (because it is interpreted).
- Not ideal for mobile app development.
- Uses more memory.
- Not suitable for applications that need **real-time speed** (like gaming engines).

6. C vs Python (Comparison Table)

Feature	C Language	Python Language
Type	Compiled language	Interpreted language
Syntax	Complex, more code needed	Simple, fewer lines of code
Speed	Very fast (close to hardware)	Slower compared to C
Memory Management	Manual (programmer handles it)	Automatic (Garbage Collection)
Use Case	System programming, OS, embedded	Web apps, AI, ML, Data Science
Learning Curve	Hard for beginners	Easy for beginners
Portability	Less portable	Highly portable

1. The Programming Cycle for Python

The steps you follow to **write and run** a Python program:

1. **Writing the Code** – Write program in .py file using an editor/IDE.
2. **Saving the Code** – Save file with .py extension (e.g., program.py).
3. **Interpreting** – Python interpreter reads code **line by line**.
4. **Execution** – Interpreter executes code and shows result/output.
5. **Debugging** – If errors occur, correct them and re-run.

Example:

Write program → Save as hello.py → Run using python hello.py → Output displayed.

2. Python IDE (Integrated Development Environment)

An **IDE** is a software that helps in **writing, testing, and running programs** easily.

- **Examples of IDEs for Python:**
 - IDLE (comes with Python)
 - PyCharm
 - Jupyter Notebook
 - VS Code
 - Spyder

Features of IDE:

- Code editor
- Syntax highlighting
- Auto-completion
- Debugging support
- Run/execute button

3. Interacting with Python Programs

Python provides **two ways** to interact:

1. Interactive Mode (Shell / REPL):

- a. Directly type and run commands in Python shell.
- b. Good for small tests.

Example:

```
>>> 5 + 3  
8
```

2. Script Mode:

- a. Write a full program in a .py file and run it.
- b. Good for larger programs.

Example:

Save file calc.py with

```
print(5 + 3)
```

Run → Output = 8.

4. Elements of Python

Basic building blocks of a Python program:

- **Identifiers:** Names given to variables, functions, classes.
Example: x, student_name.
- **Keywords:** Reserved words (cannot be used as identifiers).
Example: if, for, class, import.
- **Constants/Literals:** Fixed values.
Example: 10, "Hello", 3.14.
- **Variables:** Store data.
Example: age = 20.
- **Operators:** Symbols for operations.
Example: +, -, *, /.
- **Indentation:** Spaces used for block structure (very important in Python).
Example: if age > 18:
 print("Eligible")

5. Type Conversion in Python

Changing one data type into another.

(A) Implicit Type Conversion (Type Casting by Python automatically)

Python converts **smaller** type → **larger** type **automatically**.

Example:

```
x = 5
y = 2.5
z = x + y
print(z)    # Output 7.5
```

(B) Explicit Type Conversion (Manual using functions)

Programmer converts using functions: `int()`, `float()`, `str()`, `list()`, etc.

Example:

```
a = "100"
b = int(a)    # string → int
print(b + 50) # Output: 150
```

1. Reserved Keywords in Python

- **Definition:** Special words with fixed meaning in Python.
- They **cannot** be used as variable names, function names, or identifiers.
- Total $\approx$ 35+ keywords (may differ slightly by version).

Examples of keywords:

if, else, for, while, True, False, class, import, break, continue, def, try, except, with, return, yield.

Wrong usage:

```
for = 10  # Error! 'for' is a reserved keyword
```

2. Variables in Python

- **Definition:** A variable is a name used to store a value.
- Python is **dynamically typed** → no need to declare type.

Example:

```
x = 10      # integer
name = "GKM" # string
pi = 3.14   # float
```

Rules for Naming Variables

- ✓ Must start with a letter or underscore.
- ✓ Can contain letters, digits, underscore.
- ✓ Case-sensitive (Age ≠ age).
- ✓ Multiple assignment allowed.

Example:

```
a, b, c = 5, 10, 15
```

What Cannot Be Done

- Cannot start with a digit → `1name = 10`
- Cannot use keywords → `if = 50`
- Cannot use special symbols → `my-name = "Ram"`

3. Operators in Python

Definition: Operators are symbols used to perform operations on values or variables.

(A) Arithmetic Operators

- Perform mathematical calculations.

Operator	Meaning	Example (a=10, b=3)	Result
+	Addition	a+b	13
-	Subtraction	a-b	7
*	Multiplication	a*b	30
/	Division (float)	a/b	3.33
//	Floor division	a//b	3
%	Modulus (remainder)	a%b	1
**	Power	a**b	1000

(B) Relational / Comparison Operators

- Compare values, return **True/False**.

Operator	Meaning	Example (a=10, b=3)	Result
==	Equal to	a==b	False
!=	Not equal	a!=b	True
>	Greater than	a>b	True
<	Less than	a<b	False
>=	Greater or equal	a>=b	True
<=	Less or equal	a<=b	False

(C) Assignment Operators

- Assign values to variables.

Operator	Example	Meaning
=	x=5	Assign 5 to x
+=	x+=2	x = x + 2
-=	x-=2	x = x - 2
=	x=3	x = x * 3
/=	x/=2	x = x / 2
%=	x%=2	x = x % 2
=	x=3	x = x ** 3
//=	x//=2	x = x // 2

(D) Logical Operators

- Work with **True/False** values.

Operator	Meaning	Example (a=10, b=3)	Result
and	True if both are true	a>5 and b<5	True
or	True if one is true	a<5 or b<5	True
not	Reverses the condition	not(a>5)	False

(E) Bitwise Operators

- Work on numbers at the **bit level**.

Operator	Meaning	Example (a=6, b=3)	Result
&	AND	a & b (110 & 011)	2
	OR	a b	7
^	XOR	a ^ b	5
~	NOT (invert)	~a	-7
<<	Left shift	a<<1	12
>>	Right shift	a>>1	3

(F) Conditional / Ternary Operator

- Short way to write if-else in **one line**.

Syntax:

```
result = value_if_true if condition else value_if_false
```

Example:

```
age = 20
msg = "Adult" if age >= 18 else "Minor"
print(msg) # Output: Adult
```

(G) Membership Operators

These operators are used to **check if a value exists in a sequence (like string, list, tuple, set, dictionary)**.

- **in** → Returns True if the value is present.
- **not in** → Returns True if the value is NOT present.

Examples:

List Example

```
fruits = ["apple", "banana", "mango"]
```

```
print("apple" in fruits)      # True
print("grapes" not in fruits) # True
```

String Example

```
print("py" in "python")      # True
print("java" in "python")    # False
```

(H) Identity Operators

These operators are used to **compare the memory location of two objects** (i.e., whether they are the same object in memory).

- **is** → Returns True if both variables point to the same object.
- **is not** → Returns True if they point to different objects.

Examples:

```
x = [1, 2, 3]
y = [1, 2, 3]
z = x
```

```
print(x is z)          # True (z and x refer to same list)
print(x is y)          # False (different objects, same values)
print(x == y)          # True (values are equal)
print(x is not y)      # True (different objects)
```

1. Operator Precedence

- Operator precedence decides **which operator executes first** in an expression.
- Python follows **BODMAS rule** (Bracket → Exponent → Division/Multiplication → Addition → Subtraction).

Precedence Order (Highest → Lowest):

1. () → Parentheses
2. ** → Exponent (power)
3. *, /, //, % → Multiplication, Division, Floor Division, Modulus
4. +, - → Addition, Subtraction
5. ==, !=, >, <, >=, <= → Comparison
6. not → Logical NOT
7. and → Logical AND
8. or → Logical OR

Example:

```
result = 10 + 2 * 3 ** 2
# Step 1: 3 ** 2 = 9
# Step 2: 2 * 9 = 18
# Step 3: 10 + 18 = 28
print(result) # Output: 28
```

2. Boolean Expression

- A **Boolean expression** gives either **True** or **False**.
- It is mostly used in **conditions and loops**.

Examples:

```
print(5 > 3)          # True
print(10 == 7)        # False
print((5 < 10) and (3 > 1)) # True
print((4 > 6) or (2 < 5))  # True
print(not(5 == 5))     # False
```

3. Data Types in Python

Python has different **built-in data types**.

(A) Numeric Types

1. **int** → Integer numbers (positive, negative, zero).

```
a = 10
b = -5
print(type(a)) # <class 'int'>
```

2. **float** → Decimal numbers.

```
pi = 3.14
print(type(pi)) # <class 'float'>
```

3. **complex** → Numbers with real + imaginary part.

```
z = 3 + 5j
print(type(z)) # <class 'complex'>
```

(B) Sequence Types

1. **str (String)** → Collection of characters inside quotes.

```
name = "Python"  
print(name[0])    # P
```

2. **list** → Ordered, mutable (changeable), allows duplicates.

```
numbers = [10, 20, 30]  
numbers.append(40)  
print(numbers)    # [10, 20, 30, 40]
```

3. **tuple** → Ordered, immutable (cannot change), allows duplicates.

```
data = (1, 2, 3)  
# data[0] = 5 → Error (immutable)  
print(data)
```

4. **range** → Represents sequence of numbers.

```
r = range(1, 6)  
print(list(r))    # [1, 2, 3, 4, 5]
```

(C) Set Types

set → Unordered, mutable, no duplicates.

```
s = {1, 2, 3, 3, 4}
print(s)  # {1, 2, 3, 4}
```

(D) Dictionary (dict)

- Collection of **key:value pairs**.
- Keys must be unique, values can repeat.

```
student = {"name": "GKM", "age": 20, "course": "BTech"}
print(student["name"])  # GKM
```

(E) Boolean (bool)

- Only two values: **True** or **False**.
- Used in conditions.

```
x = True
y = False
print(type(x))  # <class 'bool'>
```

 **Notes Prepared & Organized with love & effort by Gautam Kumar Maurya (GKM)**